

NYCU-ECE DCS-2026

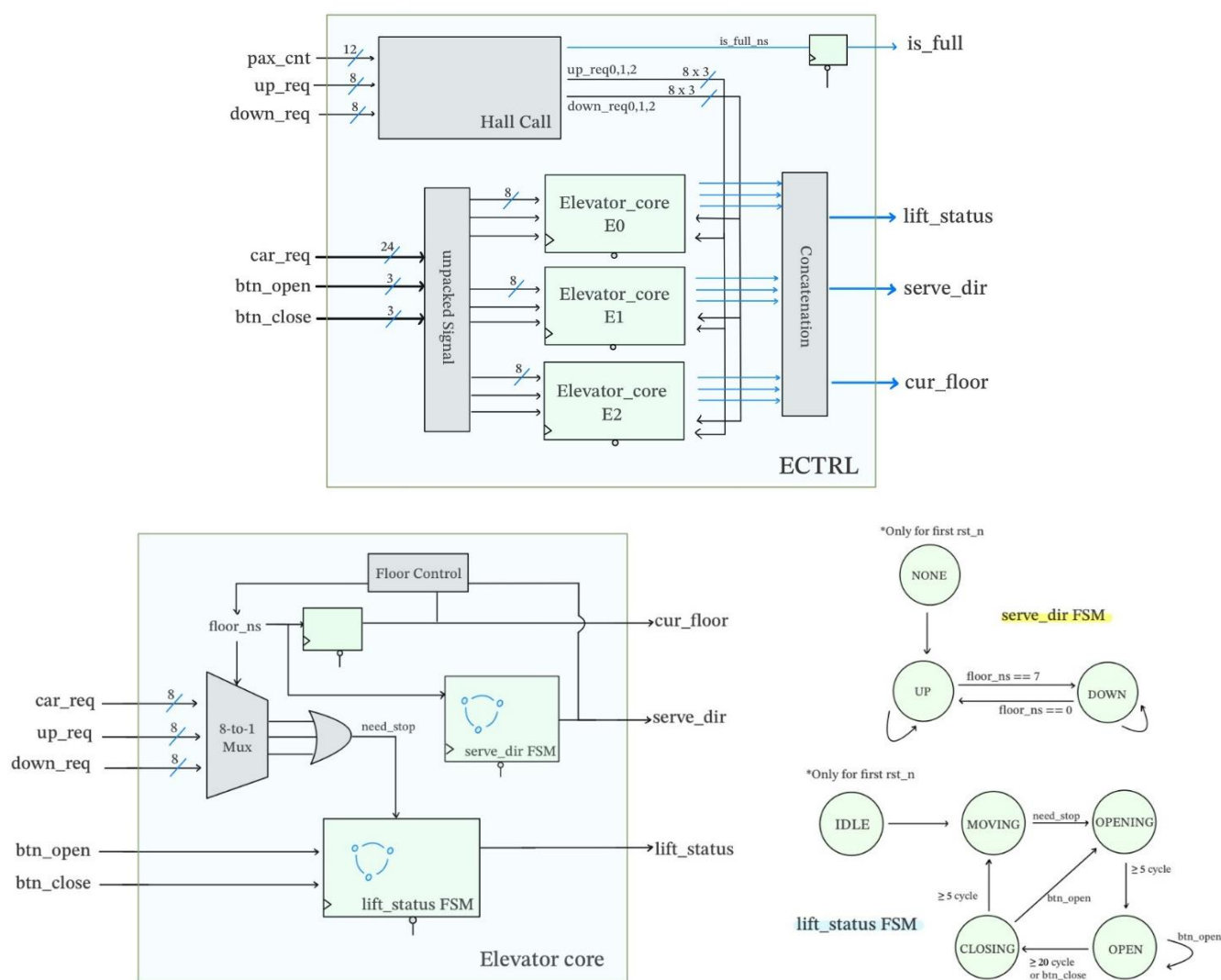
HW03 Report

Name: 楊庭瑞

Student ID: 113511266

Server Account: dcs151

Architecture



Design Methodology

在大結構 ECTRL 中，利用 hall_call module 將 up_req, down_req 分給三台電梯，盡可能避免重複分配大家去搶同一樓層，其他 input 訊號如 car_req, btn_open, btn_close 則在解包後傳入各自電梯核的模組，再由電梯核 FSM 轉換傳出 cur_floor, serve_dir, lift_status 等訊號，is_full 訊號則在最外部判斷是否超過 15 人，若超過 15 人則 hall_call 不給予分配，is_full 為 output 訊號，因此一定要使用 FF 來存值，否則 input output external delay 會導致 timing violated。

Optimization of Performance :

1. **電梯運行 FSM 邏輯:** 最初我利用接近生活的概念去想電梯邏輯，電梯沒有請求任務時就進入 IDLE，後來改成永動機，一直從上到下下到上來回接客的方式，可以大幅減少 FSM 的判斷邏輯，除了最開始的 rst，其他都不進入 IDLE，讓面積縮了蠻多。
2. **Latency 與 Area 的 Trade-off:** 原先在 top module 是對每台電梯接同向請求，且加入遮罩(當電梯上行時，不接收當前樓層之下的請求)，這樣的方法雖然可以讓 latency 小了很多，但是面積偏大，後來發現若只對第一個電梯放遮罩，其他電梯接同向請求，latency 雖然會大一點，但是面積可以變小蠻多，極為捨棄速度換取面積縮減。
3. **簡化判斷式:** 透過簡化一些邏輯判斷，可以讓面積縮小很多，像是 timer 是從 0 遞增加上去的 `timer_cs < 4` 判斷改為 `!timer_cs[2]`，可以減少比較器，面積小了約 300。
4. **將不必要的 rst_n 移除:** 由於在第一次的 rst_n, state 必須要進入 IDLE，我便將 state 在 idle 的時候把 timer_ns 設為 0，這樣就可以讓 timer 不用使用有 rst 功能的 DFF。

Discussion and Reflection

本次作業的設計空間很大，同一個問題可以對應多種 scheduling 與架構實現方式。如何權衡電梯的分配就很難，追求極致的分配演算法就會導致判斷邏輯多，面積大，若分配不恰當，面積小但 latency 會暴增甚至會超過 2000 cycle 直接 fail，導致自己花了很多時間尋找最佳 performance，再依賴 AI 的情況下，很能找出 performance 的最佳解，AI 解法多數比較完整、對稱，想不出那種給前兩台優先分配剩下的丟給第三台，或是只給第一台上遮罩這種調度邏輯，AI 工具雖然能大幅增加效率，但一定要加入自己的理解，從中引導，才有機會得到最佳結果。總體而言這次的作業讓我學到了很多，也更明白該怎麼權衡 latency vs area 的關係，題目也蠻有趣的。

AI-use Log

If AI was used, please include the following details:

Copilot Agent, GPT-5.3-Codex

<https://chatgpt.com/share/69e62aae-9d3c-83a7-a7e7-3be674d7b54a>

- Key Prompts: 3–5 primary prompts (paste the screenshots).

```
always_comb begin
    st_idle   = 1'b0;
    st_moving = 1'b0;
    st_opening = 1'b0;
    st_open   = 1'b0;
    st_closing = 1'b0;

    case (state_cs)
        IDLE: st_idle   = 1'b1;
        MOVING: st_moving = 1'b1;
        OPENING: st_opening = 1'b1;
        OPEN: st_open   = 1'b1;
        CLOSING: st_closing = 1'b1;
        default: /* all remain 0 */;
    endcase
end

end // 為甚麼這樣寫的面積會比我直接用 assign st_idle = (state_cs == IDLE) 還要小
```

```
assign up_mask  = (8'hFF << floor_e[0]) & 8'h7F; //
bit[7]=0: 7F 無 up_req
assign down_mask = ~(8'hFE << floor_e[0]) & 8'hFE; //
bit[0]=0: 0F 無 down_req
```

- 遮罩邏輯用case寫會比較小嗎

Hall call 拉一個模組出來會變大還是變小，哪種方法比較建議

- 議